metric internship – washington

June 29, 2026

What you'll be able to do

By the end of this tutorial you can:

- explain the core loop: the model decides, your code does;
- tell tool calling and MCP apart – they live on different sides of your app;
- choose between direct orchestration, tool calling, MCP, and skills for a task;
- recognise and avoid the common traps in each.

Outline

The problem (and our running example)

An LLM only produces text. By itself it cannot know today's weather, the latest news, or live scores.

One question, all the way through

We will follow a single request through every approach:

"Will it rain in Yerevan today?"

The model has no live data. Something outside it must fetch that – the questions are *who orchestrates the fetch* and *where the fetcher lives*.

What actually happens (the gist)

For "Will it rain in Yerevan today?":

1. the app sends the question to the model;
2. the model can't answer alone, so it asks for a function: `get_weather(city=''Yerevan'')`;
3. your code runs `get_weather` – it hits a weather API;
4. the result goes back to the model;
5. the model writes the answer: "No rain expected; clear, 20C."

That five-step shape is the heart of everything below. Now let's name the idea behind it.

The one idea behind everything

Mental model

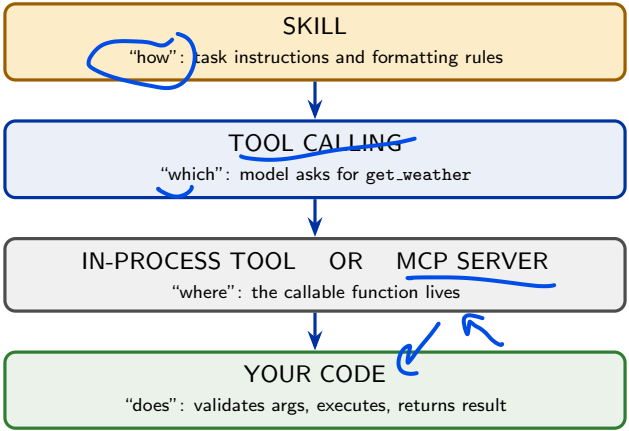
The model decides, your code does. The model never runs code – it emits a structured request; your program executes it and feeds the result back.

The four approaches differ in three questions:

1. **Who decides** what runs – your code, or the model?
2. **Where do the tools live** – in-process, or a separate server?
3. **What is loaded up front** – everything, or only what's needed?

Layer map before the details

Keep these four layers separate:



Each section changes one layer. That is the safest way to avoid mixing up tools, MCP, and skills.

One system, four upgrades

Version	What changed from the previous version?
Direct orchestration	Your code routes the request and calls the function itself.
Tool calling	The model now chooses from a tool menu; your code still executes.
MCP	The same tool moves to a separate server; your app bridges it back to tool calling.
Skills	Task instructions load on demand before the model uses the relevant tool.

Reading strategy

Do not memorize four labels. Ask: who chooses? where does the function live? what instructions were loaded?

Roadmap

1. Direct orchestration
2. Tool calling
3. MCP
4. Skills (SKILL.md)
5. Choosing

Direct orchestration — the idea

What it is

Your code owns the control flow. You classify the request, call the right function yourself, then (optionally) use the LLM only to phrase the answer. The model does not choose tools.

For our example: *your* code detects "weather", extracts "Yerevan", calls `get_weather`, and asks the LLM to phrase the result.

Good when

- few, known intents
- you want determinism & low cost
- auditable, testable flow

Bad when

- many/open-ended intents
- the routing logic balloons
- you reinvent what tool calling gives free

Direct orchestration — code

```
1  def answer(question: str) -> str:
2      q = question.lower()
3      if "weather" in q or "rain" in q or "umbrella" in q:
4          city = extract_city(question) # regex, form field, or parser
5          data = get_weather(city) # <== KEY: your code picks the tool
6          return llm_phrase(question, data)
7      if "latest" in q or "today" in q:
8          hits = web_search(question)
9          return llm_phrase(question, hits)
10     return llm_answer(question) # general knowledge
```

Key line (5): *your* code decides to call `get_weather` – the model is not involved in the choice. Predictable, but every new capability is another branch.

Direct orchestration — exact trace

```
1. user -> app:
   "Will it rain in Yerevan today?"

2. app routes locally:
   q contains "rain" -> choose weather branch

3. app extracts arguments:
   city = "Yerevan"

4. app runs code directly:
   data = get_weather("Yerevan")

5. app asks model only to phrase:
   llm_phrase(question, data)

6. assistant -> user:
   "No rain expected in Yerevan today..."
```

The model never chooses a tool here. The decision point is step 2, inside your code.

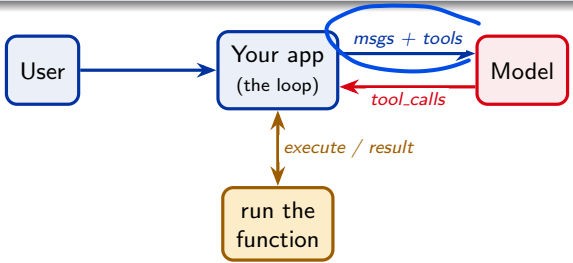
Roadmap

1. Direct orchestration
2. Tool calling
3. MCP
4. Skills (SKILL.md)
5. Choosing

Tool calling — the idea

What it is

You hand the model a menu of tools (JSON schemas). The model replies "call `get_weather(city='Yerevan')`". Your code runs it and feeds the result back. The model decides *which* tool and *when*.



Every arrow is one HTTP request. A regular answer is just the text path on round 1.

Tool calling — first, the transcript

Before SDK code, understand the message order:

```
user:
  Will it rain in Yerevan today?

assistant:
  tool_call id=call_123 name=get_weather args={"city":"Yerevan"}

tool:
  tool_call_id=call_123
  {"condition":"clear sky","temperature_c":20,"wind_kmh":7}

assistant:
  No rain expected in Yerevan today. It is clear and about 20C.
```

The assistant's tool-call message must be in history before the tool result, and the result must echo the same `tool_call_id`. (Args shown decoded – on the wire `arguments` is a JSON string.)

Tool calling — code (OpenRouter)

```python
client = OpenAI(base_url="https://openrouter.ai/api/v1",
                api_key=os.environ["OPENROUTER_API_KEY"])

tools = [{"type": "function", "function": {
    "name": "get_weather",
    "description": "Current weather for a city.",
    "parameters": {"type": "object",
       "properties": {"city": {"type": "string"}},
       "required": ["city"]}}}]

resp = client.chat.completions.create(model=MODEL, messages=msgs, tools=tools)
msg = resp.choices[0].message
msgs.append(msg.model_dump()) # assistant message with tool_calls
for call in msg.tool_calls or []:
    args = json.loads(call.function.arguments) # <== KEY: arguments is a STRING
    result = get_weather(**args) # your code executes
    msgs.append({"role": "tool", "tool_call_id": call.id,
                 "content": json.dumps(result)}) # id must match
```

Key line (15): `arguments` arrives as a JSON *string* – always `json.loads` it. And every `tool_call_id` must echo the call's id.

Tool calling — gotchas

- `arguments` is a JSON string, not a dict – always `json.loads` it (and guard `""`).
- You must append the assistant message with `tool_calls` before the tool results.
- Every `tool_call_id` needs exactly one matching `tool` message (watch parallel calls).
- The model can hallucinate tool names/args – validate before calling `**args`.
- Cap the rounds – tools can loop forever.
- Cost: the whole tool list is re-sent every round and counts as input tokens.

Roadmap

1. Direct orchestration
2. Tool calling
3. MCP
4. Skills (SKILL.md)
5. Choosing

MCP — the idea

What it is

A standard protocol for exposing tools from a separate server so any app can plug them in ("USB-C for AI"). It does not replace tool calling – the model still chooses via tool calling; MCP just standardises where tools live.

Primitives: tools (do), resources (read), prompts (templates).

MCP — server + bridge to OpenRouter

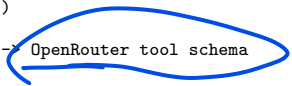
```python
# server.py -- a standalone tool server (FastMCP)
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("weather")

@mcp.tool() # <== KEY: turns a function into a tool
def get_weather(city: str) -> dict:
    """Current weather for a city.""" # docstring+hints -> JSON Schema
    return weather.get_weather(city)
mcp.run() # stdio transport
```

```python
# client: MCP inputSchema is ALREADY JSON Schema -> near no-op conversion
def mcp_tool_to_openai(t):
    return {"type": "function", "function": {
        "name": t.name, "description": t.description,
        "parameters": t.inputSchema}} # <== KEY: already JSON Schema
# list_tools() -> convert -> normal OpenRouter loop -> session.call_tool(name, args)
```

Key idea: the bridge is nearly free – MCP's `inputSchema` is already the JSON Schema that tool calling wants.

MCP — exact trace

```
SETUP: host connects to the tool server
  1. host starts mcp_server.py over stdio
  2. host creates ClientSession(read, write)
  3. host awaits session.initialize()
  4. host calls session.list_tools()
  5. host converts each inputSchema -> OpenRouter tool schema

USER TURN: normal tool-calling loop
  6. host sends messages + converted tools to the model
  7. model emits get_weather({"city":"Yerevan"})
  8. host routes that call to session.call_tool(...)
  9. MCP server runs weather.get_weather(...)
 10. host appends the tool result and asks the model for final text
```



The model only sees ordinary tool calling; MCP is hidden behind the Python host.

MCP — plug in a server you didn't write

Because MCP is a standard, you can point the same bridge at someone else's server:

```
# our own server:
SERVER = StdioServerParameters(command=sys.executable, args=["mcp_server.py"])

# a third-party server from the web (official "fetch" server, no API key):
SERVER = StdioServerParameters(command="uvx", args=["mcp-server-fetch"])
```

Nothing else changes – `list_tools` → convert → loop is identical. The model now gets a `fetch` tool from code we never wrote:

```
user: Fetch https://example.com and give the heading.
model: fetch({"url":"https://example.com"}) # tool from a third-party server
tool: "Example Domain ... for use in documentation examples ..."
model: The heading is "Example Domain".
```

The payoff of MCP: reuse existing tool servers instead of writing your own.

The key question: tool calling vs MCP

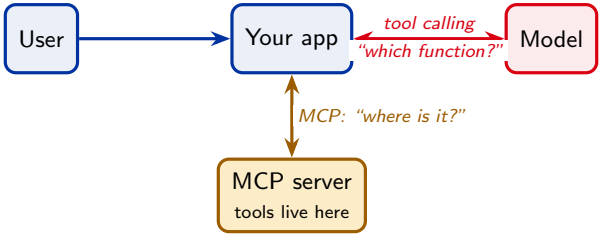
They are on different sides of your app – not alternatives.

Tool calling = app ↔ model

How the model asks your app for a function. Built into the LLM API. Works with plain Python functions and zero MCP.

MCP = host/app ↔ tool server

How a host discovers & calls tools in another process, vendor-neutrally. The model sees only the converted tool schemas.



MCP tools reach the model *through* tool calling. MCP buys reuse & ecosystem, not a new decision mechanism.

Roadmap

1. Direct orchestration
2. Tool calling
3. MCP
4. Skills (SKILL.md)
5. Choosing

Skills — the idea

What it is

A folder with a `SKILL.md` (instructions) + optional scripts/files, loaded on demand. It packages *know-how*, not capabilities. (Anthropic feature; OpenRouter has no native skills – you port the *pattern*.)

The key distinction

A tool lets the assistant *do* something. A skill tells the assistant *how* to do it well.

Progressive disclosure (the whole point)

Level 1	name + description	always (~100 tokens each)
Level 2	full SKILL.md body	only when triggered
Level 3	bundled files/scripts	only when referenced

100 skills cost ~100 tokens each up front; heavy content loads only when needed.

Skills — what changes in the answer

```
without a skill:
  model -> get_weather("Yerevan") -> generic weather answer

with weather-lookup skill:
  model -> load_skill("weather-lookup")
        -> read formatting rules
        -> get_weather("Yerevan")
        -> yes/no first, then humidity and wind
```

The capability is still `get_weather`. The skill changes the procedure and output standard.

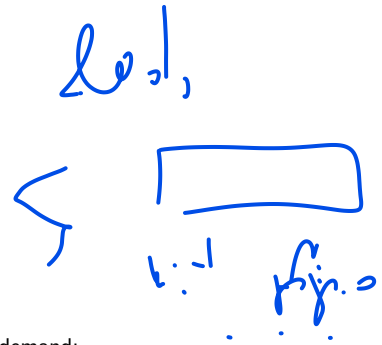
Strict skills — exact trace

```
START OF TURN: only one tool is visible
  tools=[load_skill]
  system prompt lists skill names + descriptions only

1. user -> app:
   "Should I bring an umbrella in Yerevan?"

2. model -> app:
   load_skill({"name":"weather-lookup"})

3. app -> model as tool result:
   full SKILL.md body
   [Tools now available: get_weather]

4. next model round sees:
   tools=[load_skill, read_skill_file, get_weather]

5. model -> app:
   get_weather({"city":"Yerevan"})

6. model writes final answer using the loaded instructions
```

Strict mode forces the path: instruction first, capability second. It still cannot guarantee perfect adherence.

SKILL.md — format + porting to OpenRouter

```
---
name: weather-lookup
description: Fetches weather and formats it. Use when the user asks about
  weather, temperature, rain, "should I bring an umbrella", ...
allowed-tools: [get_weather] # which tools this skill needs
---
# Weather Lookup
1. Call get_weather with the city. 2. Lead with one plain sentence. ...
```



Porting the pattern (no native support):

- put only each skill's name+description in the system prompt;
- give the model a `load_skill(name)` tool that returns the full body on demand;
- Flat = domain tools always available (model may skip `load_skill`). Strict = tools *locked* until a skill unlocks them (forces the path).

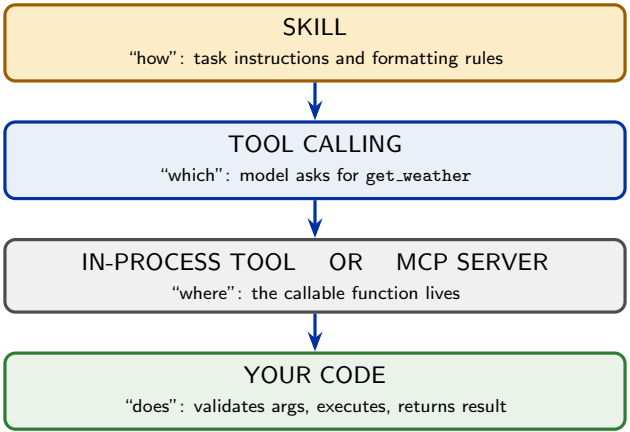
Skills — gotchas

- Vague descriptions never trigger – the description is the entire match surface. Be specific, third-person, with the words a user would say.
- Enforcement $\neq$ adherence: you can force `load_skill` to run, but a weak model may still ignore the instructions it loaded.
- Specific instructions can still fail – e.g. "do not add a year to search queries" may be ignored by a weak model.
- Skills are instructions, not magic – effectiveness is capped by the model.
- Keep `SKILL.md` small; push detail into bundled files (loaded on demand).
- Security: bundled scripts run with your permissions – audit them.

Roadmap

1. Direct orchestration
2. Tool calling
3. MCP
4. Skills (SKILL.md)
5. Choosing

They are layers, not competitors



Same four layers as the start. A skill sets *how*; tool calling picks *which*; the function lives in-process or over *MCP*; and *your code* does the work. Each approach changes one layer – direct orchestration just skips the model's choice.

The same question, four ways

"Will it rain in Yerevan today?"

- **Direct orchestration:** *your code* detects "weather", calls `get_weather(''Yerevan'')`, LLM phrases.
- **Tool calling:** the *model* emits `get_weather(city=''Yerevan'')`; your code runs it; the model answers.
- **MCP:** same model-facing tool call, but `get_weather` lives in a separate server; the host routes the call there.
- **Skills:** the model loads the `weather-lookup` skill (formatting rules), *then* calls `get_weather`, and answers in the skill's style.

Same five-step shape; what changes is who decides and where the tool lives.

Side-by-side comparison

	Direct orch.	Tool calling	MCP	Skills
Who decides	code	model	model	model (picks the skill)
Tools live	in-process	in-process	sep. server	n/a (instructions)
Loaded up front	all logic	all schemas	all schemas	only descriptions
Setup cost	low	low	high	medium
Reuse across apps	no	no	yes	yes (portable md)
Best for	few/known flows	open-ended choice	shared tools	many instruction sets

All four can co-exist in one assistant.

Decision guide — when to use which

- Few, well-known flows; want determinism → **direct orchestration**.
- Open-ended; let the model pick among tools → **tool calling** (start here for most assistants).
- Reuse tools across apps / use the ecosystem → **MCP** (otherwise plain functions are simpler).
- Many task-specific instruction sets; context is filling up → **skills**.

Pragmatic path

Build with plain tool calling first. Add MCP only when a tool needs to be shared. Add skills only when the system prompt gets bloated. Use direct orchestration for the narrow, must-be-correct paths.

Bridge to text-to-SQL

The same architecture shows up in the internship SQL assistant:

	Weather toy	**Text-to-SQL**
Tool	`get_weather(city)`	`run_sql(query)`
Direct orchestration	code detects weather	code runs classify → generate SQL → execute/repair
Tool calling / MCP	model asks for weather	model asks to run SQL and can recover from results/errors
Skill	weather answer style	SQL procedure, safety rules, answer style

Important implication

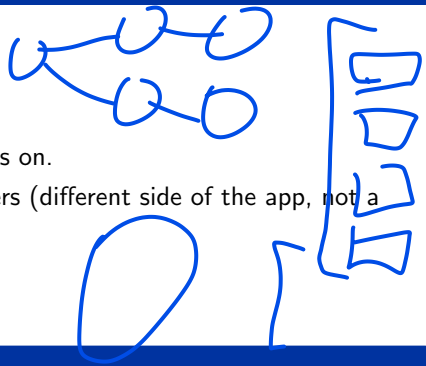
For SQL, the quality lift usually comes from the execution-feedback loop (run → inspect result/error → repair), not from MCP by itself.

Cross-cutting engineering footnotes

- Logging silently broken: a later `basicConfig` is a no-op if the root logger is already configured (use `force=True`).
- Trusting docs over the installed version: an SDK class in the docs may not exist in the release – introspect what's installed.
- Forgetting to cache a geocode/weather call $\rightarrow$ hammering a free tier.
- Letting one bad turn crash the REPL – wrap each turn; keep the session alive.

Summary

- One idea: model decides, your code does.
- **Direct orchestration** – code routes; simple, rigid.
- **Tool calling** – model routes; the foundation everything builds on.
- **MCP** – tool calling + a standard plug for reusable tool servers (different side of the app, not a replacement).
- **Skills** – on-demand instructions via progressive disclosure.



Rule of thumb

Start with tool calling. Add MCP only when a tool must be shared across apps. Add skills only when the system prompt gets bloated. Use direct orchestration for the narrow, must-be-correct paths.

Questions?